Cyber Security

HTTP, HTTPS, COOKIES

By
Dr. Mudassar Raza
Professor
Department of Computer Science
Namal University Mianwali

https://www.slideshare.net/slideshow/hyper-text-transfer-protocol-http/27133041

HTTP

- HTTP stands for HyperText Transfer Protocol.
- It is a protocol used in TCP/IP Application layer to access the data on the World Wide Web (www).
- The HTTP protocol can be used to transfer the data in the form of (Multipurpose Internet Mail Extensions) MIME-like format such as plain text, hypertext, audio, video, and so on.
- This protocol is known as HyperText Transfer Protocol because of its efficiency that allows us to use in a hypertext environment where there are rapid jumps from one document to another document.

Features of HTTP

- **Connectionless protocol:**

HTTP client initiates a request and waits for a response from the server. When the server receives the request, the server processes the request and sends back the response to the HTTP client after that the client disconnects the connection. The connection between client and server exist only during the current request and response time only.

- **Media independent:**

HTTP protocol is a media independent as data can be sent as long as both the client and server know how to handle the data content. It is required for both the client and server to specify the content type in MIME-type header.

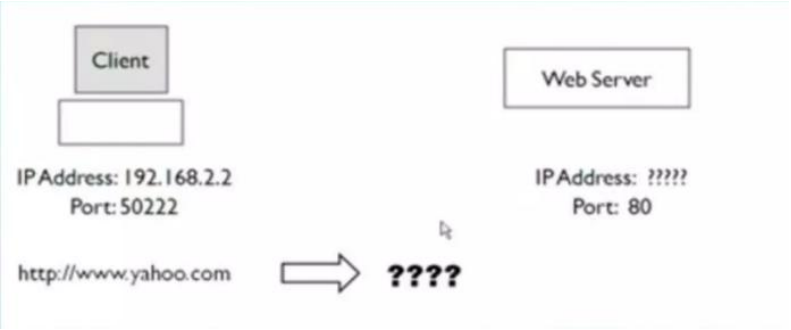
- **Stateless:**

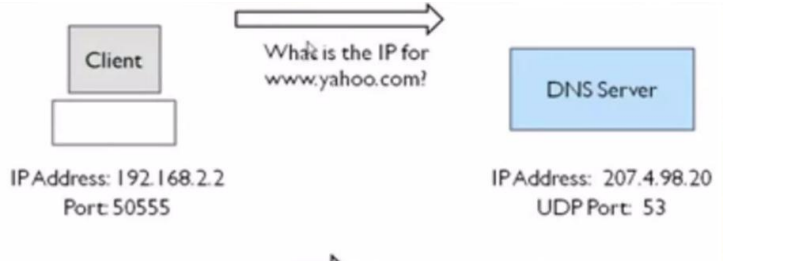
HTTP is a stateless protocol as both the client and server know each other only during the current request. Due to this nature of the protocol, both the client and server do not retain the information between various requests of the web pages.

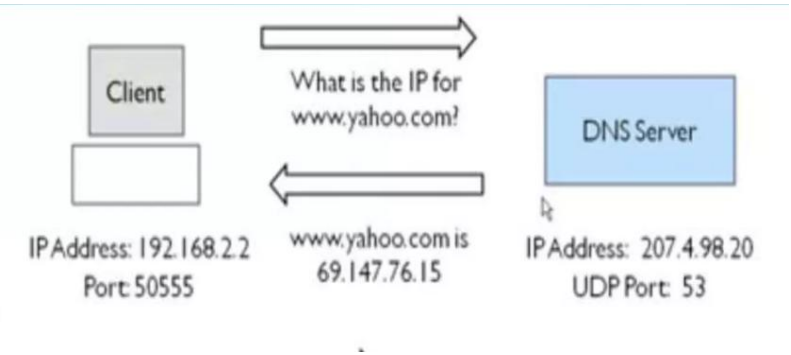
HTTP Request /Response cycle

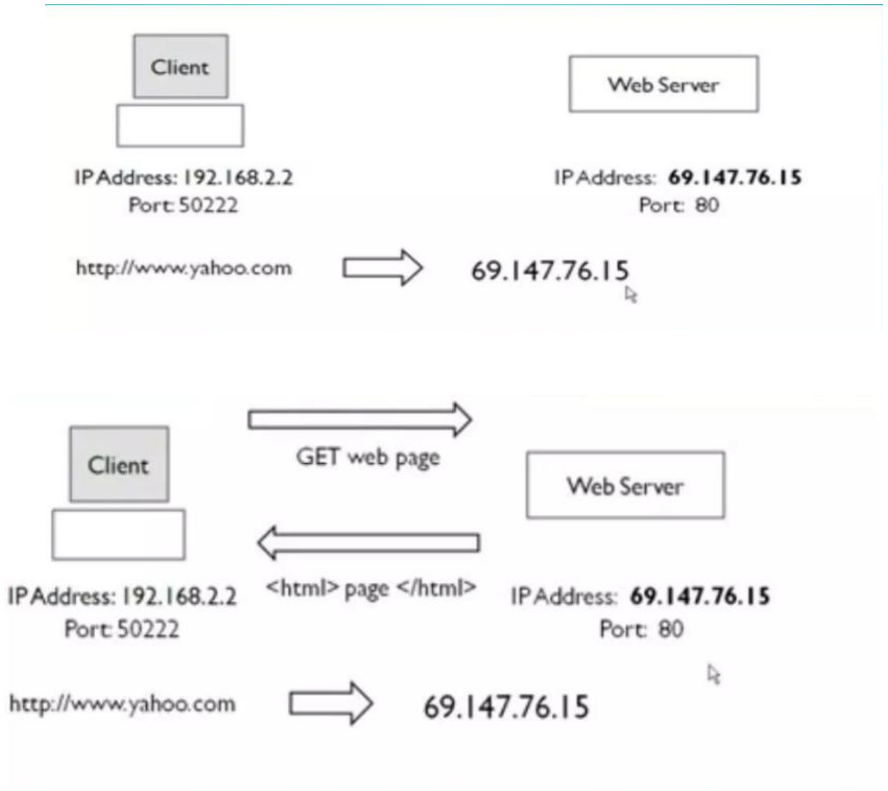
- Communication between clients and servers is done by requests and responses
- 1.A client (a browser) sends an HTTP request to the web server
- 2. A web server receives the request
- 3.The server runs an application to process the request
- 4.The server returns an HTTP response (output) to the browser
- 5.The client (the browser) receives the response

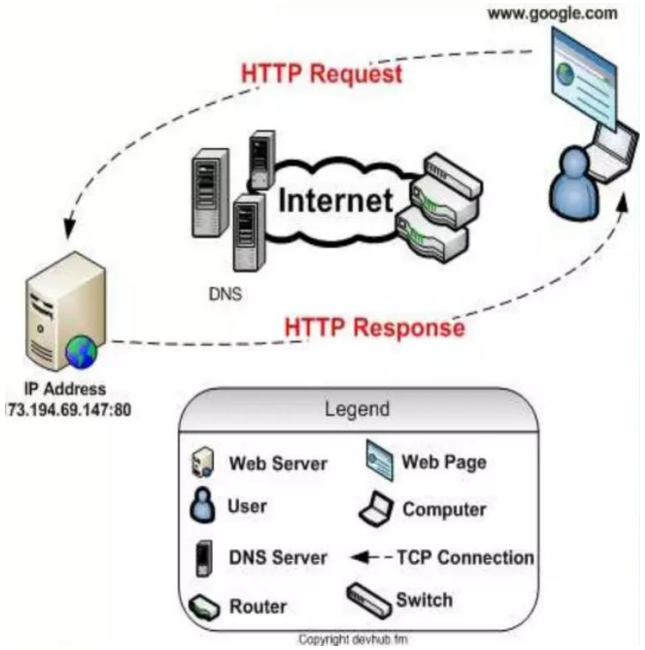
HTTP Request /Response cycle











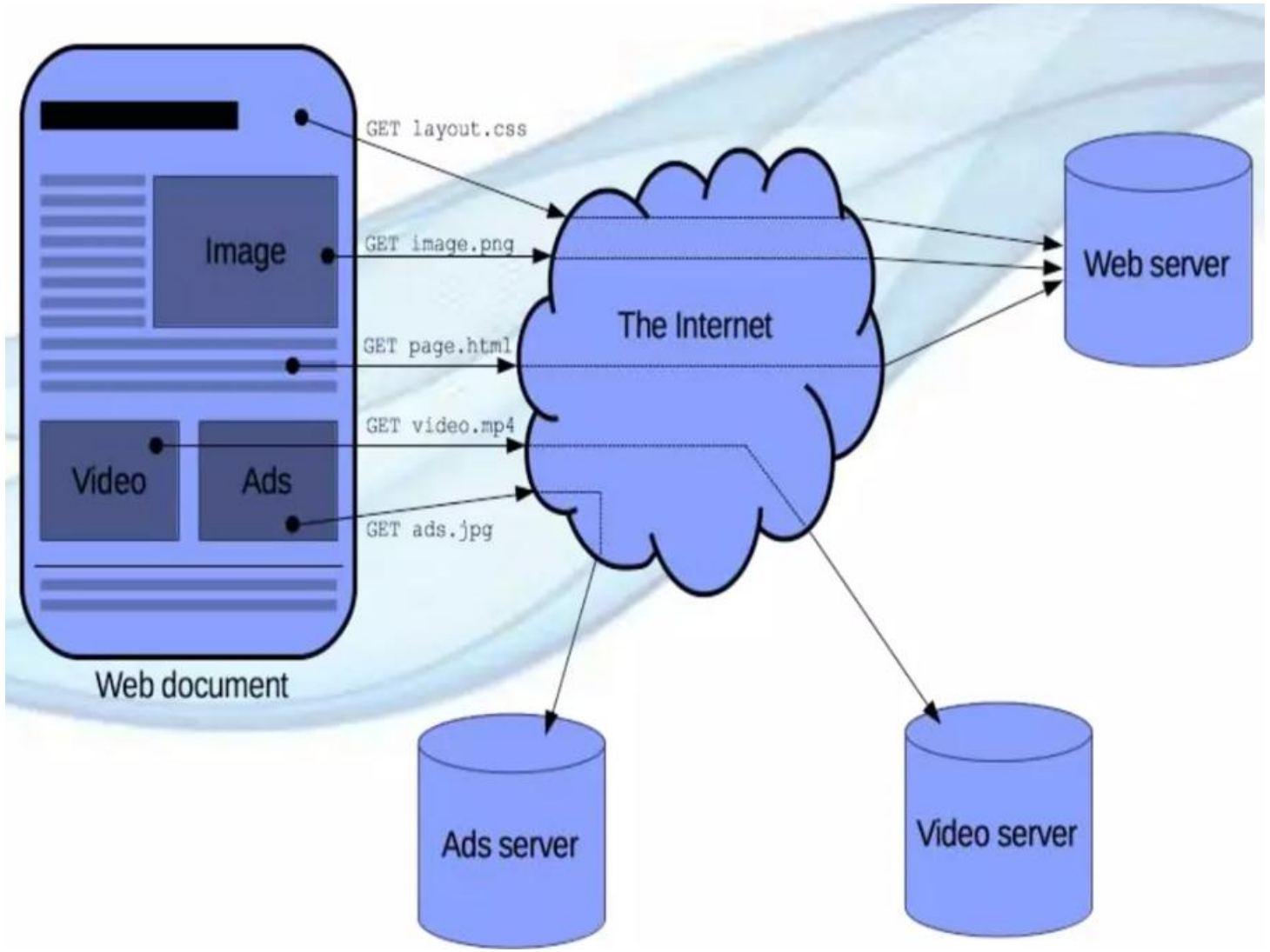
HTTP request /responseto browse a website

- A website is made up of many different files, which are sored in a server or in varies servers. These files come in two main types:
- **Code files:** Websites are built primarily from HTML, CSS, JavaScript, php,…etc.
- **Assets:** This is a collective of all the other contents of website, such as images, music, video, Word documents,

HTTP request /responseto browse a website

- HTTP makes several request to browse a website
- 1.The browser requests an HTML page. The server returns an HTML file.
- 2.The browser requests a style sheet. The server returns a CSS file.
- 3.The browser requests an JPG image. The server returns a JPG file.
- 4.The browser requests JavaScript code. The server returns a JS file
- 5.The browser requests data. The server returns data (in XML or JSON).

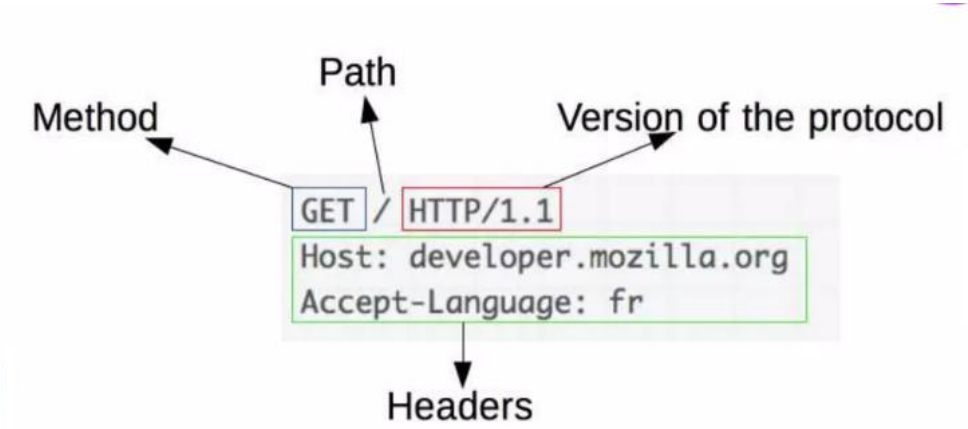
HTTP request /responseto browse a website



HTTP Request

Requests consistsof the following elements:

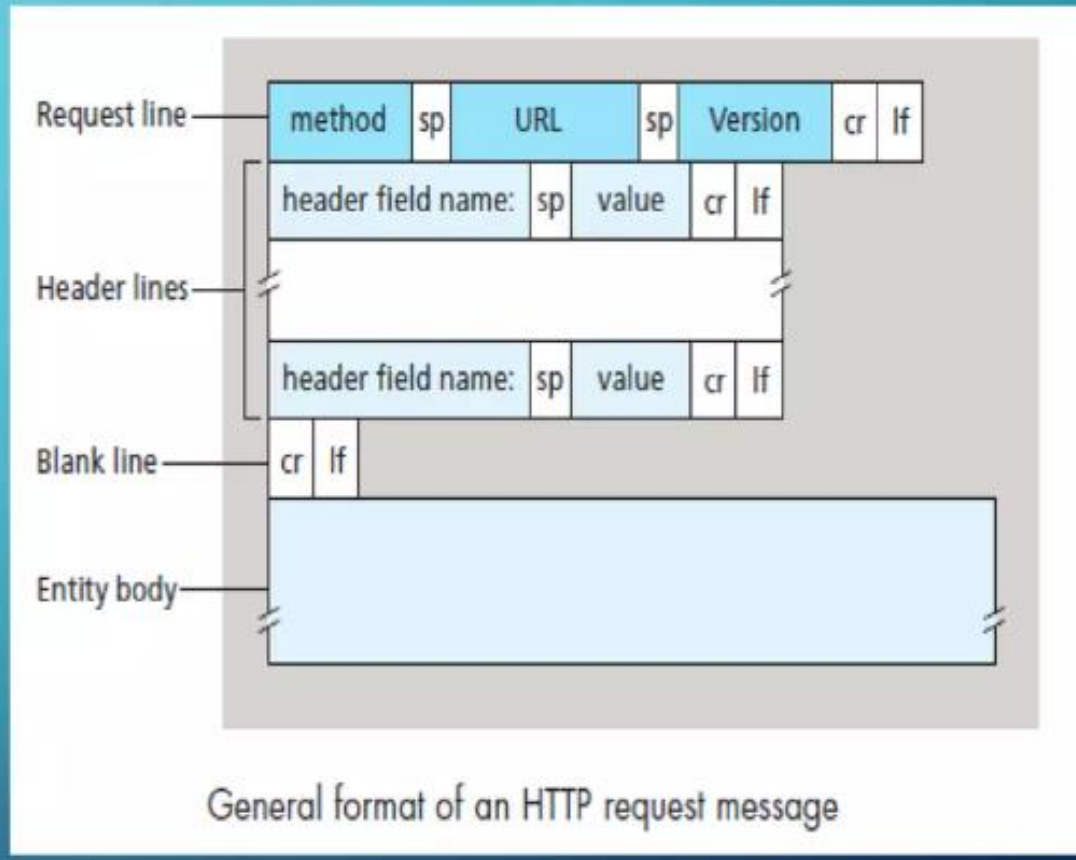
- **Method**, HTTP methods like GET, POST, OPTIONS or HEAD that defines the operation the client wants to perform.
- **Path** of the resource to fetch; the URL of the resource, for example without the protocol (http://), the domain (here, developer.mozilla.com).
- **Version** of the HTTP protocol.
- **Optional headers** that convey additional information for the servers.
- **Or a body**, for some methods like POST, similar to those in responses, which contain the resource sent.



HTTP Request

HTTP REQUEST MESSAGE

- The first line of an HTTP request message is called the **request line**; the subsequent lines are called the **header lines**. The request line has three fields: the method field, the URL field, and the HTTP version field. The method field can take on several different values, including GET, POST, HEAD, PUT, and DELETE etc. The great majority of HTTP request messages use the GET method. The GET method is used when the browser requests an object, with the requested object identified in the URL field.



General format of an HTTP request message

HTTP Request

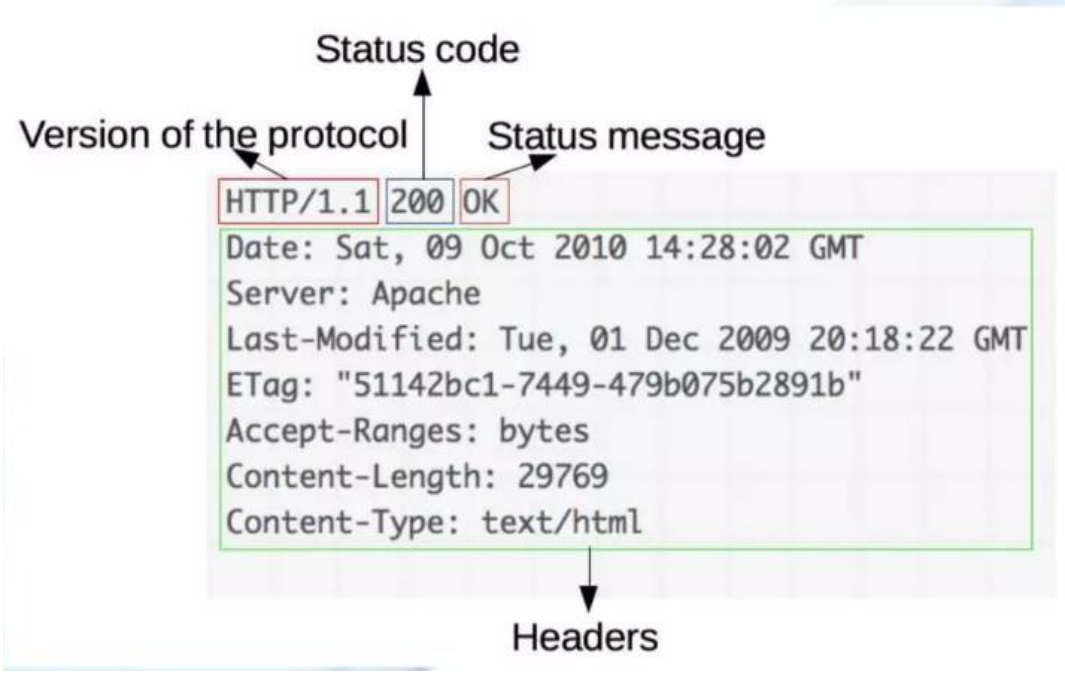
REQUEST METHODS

- GET: Retrieve Document identified in URL
- HEAD: Retrieve meta information about document identified in URL
- DELETE: Delete specified URL
- OPTIONS: Request information about available options
- PUT: Store document under specified URL
- POST: Give information to server
- TRACE: Loopback request message
- CONNECT: For use by Proxies

HTTP Response

Responses consist of the following elements:

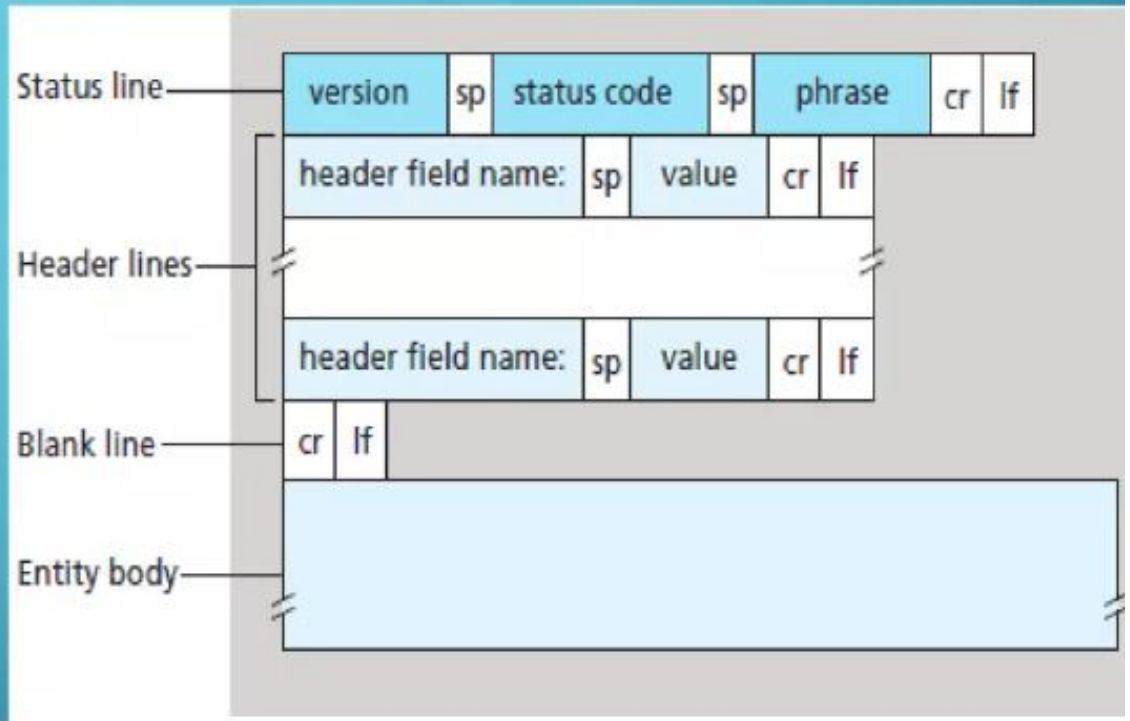
- The **version** of the HTTP protocol they follow.
- A **status code**, indicating if the request was successful, or not, and why.
- A **status message**, a non-authoritative short description of the status code.
- **HTTP headers,** like those for requests.
- Optionally, a body containing the fetched resource.



HTTP Response

HTTP RESPONSE MESSAGES

- It has three sections: an initial **status line**, **header lines**, and then the **entity body**. The entity body contains the requested object itself. The status line has three fields: the protocol version field, a status code, and a corresponding status message.



General format of an HTTP response message

HTTP Response

SOME COMMON STATUS CODES AND ASSOCIATED PHRASES

- Some common status codes and associated phrases include:
  - 200 OK: Request succeeded and the information is returned in the response.
  - 301 Moved Permanently: Requested object has been permanently moved; the new URL is specified in Location: header of the response message. The client software will automatically retrieve the new URL.
  - 400 Bad Request: This is a generic error code indicating that the request could not be understood by the server.
  - 404 Not Found: The requested document does not exist on this server.
  - 505 HTTP Version Not Supported: The requested HTTP protocol version is not supported by the server.

Code	Type	Example Reasons
1xx	Informational	Request received, continuing process
2xx	Success	Action successfully received, understood, and accepte
3xx	Redirection	Further action must be taken to complete the request
4xx	Client error	Request contains bad syntax or cannot be fulfilled
5xx	Server error	Server failed to fulfill an apparently valid request

Five types of HTTP result codes.

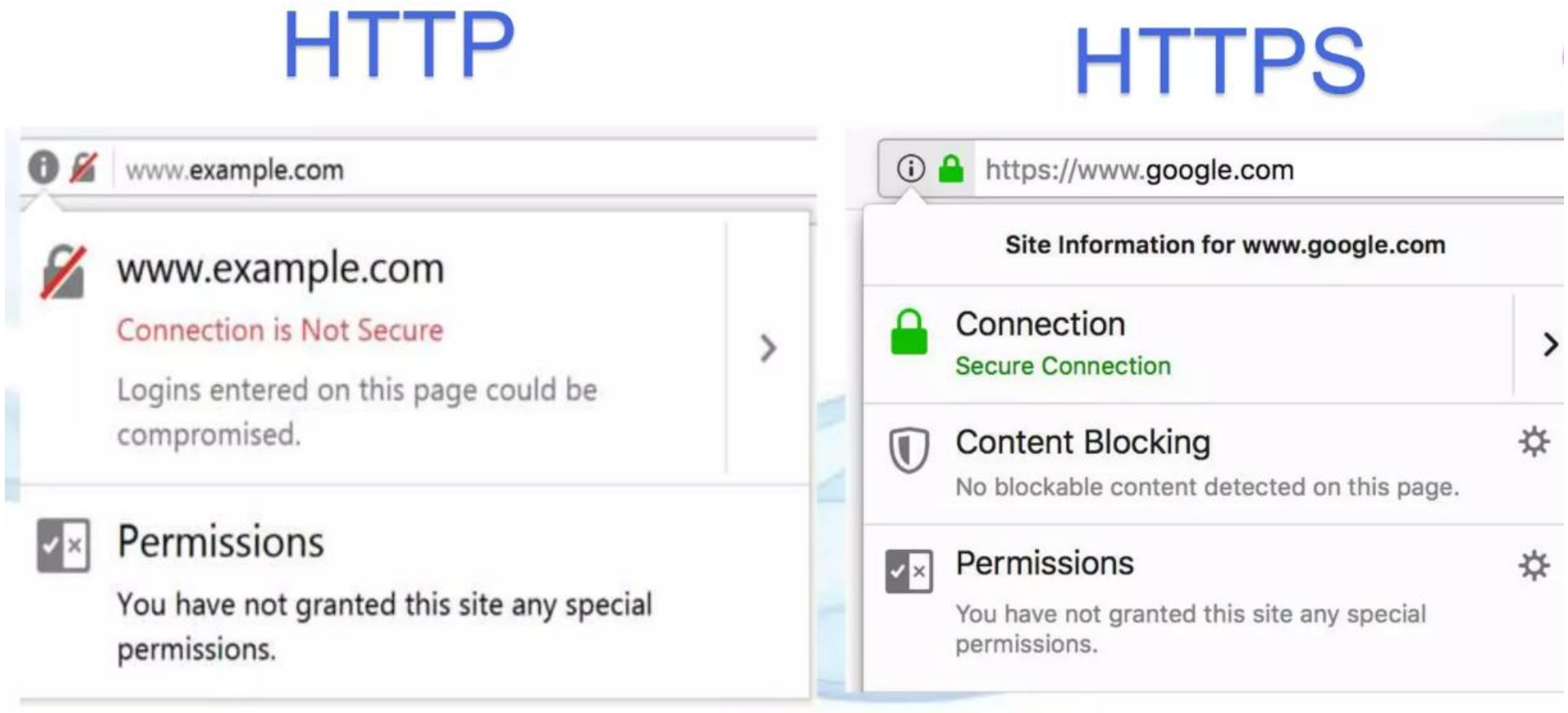
HTTPS

- HTTPS is Hypertext Transfer Protocol Secure.
- The HTTP protocol does not provide the security of the data, while **HTTPS ensures the security of the data.** Therefore, we can say that HTTPS is a secure version of the HTTP protocol.
- **This protocol allows transferring the data in an encrypted form**. The use of HTTPS protocol is mainly required where we need to enter the bank account details. The HTTPS protocol is mainly used where we require to enter the login credentials. In modern browsers such as chrome, both the protocols, i.e., HTTP and HTTPS, are marked differently.
- To provide encryption, HTTPS uses an encryption protocol known as **Transport Layer Security (TSL),** and **Secure Sockets Layer (SSL).** This protocol uses a mechanism known as asymmetric public key infrastructure, and it uses two different keys which are given below:
  - Private key: This key is available on the web server, which is managed by the owner of a website. It decrypts the information which is encrypted by the public key.
  - Public key: This key is available to everyone. It converts the data into an encrypted form.

HTTP VS HTTPS

HTTP	HTTPS
The full form of HTTP is the Hypertext Transfer Protocol.	The full form of HTTPS is Hypertext Transfer Protocol Secure.
It is written in the address bar as http://	It is written in the address bar as https://
The HTTP transmits the data over port number 80.	The HTTPS transmits the data over port number 443.
It is unsecured as the plain text is sent, which can be accessible by the hackers.	It is secure as it sends the encrypted data which hackers cannot understand.
It is mainly used for those websites that provide information like blog writing.	It is a secure protocol, so it is used for those websites that require to transmit the bank account details or credit card numbers.
It does not use TSL and SSL.	It uses TSL and SSL that provides the encryption of the data.
Google does not give the preference to the HTTP websites.	Google gives preferences to the HTTPS as HTTPS websites are secure websites.
The page loading speed is fast.	The page loading speed is slow as compared to HTTP because of the additional feature that it supports, i.e., security.

HTTPS vsHTTP

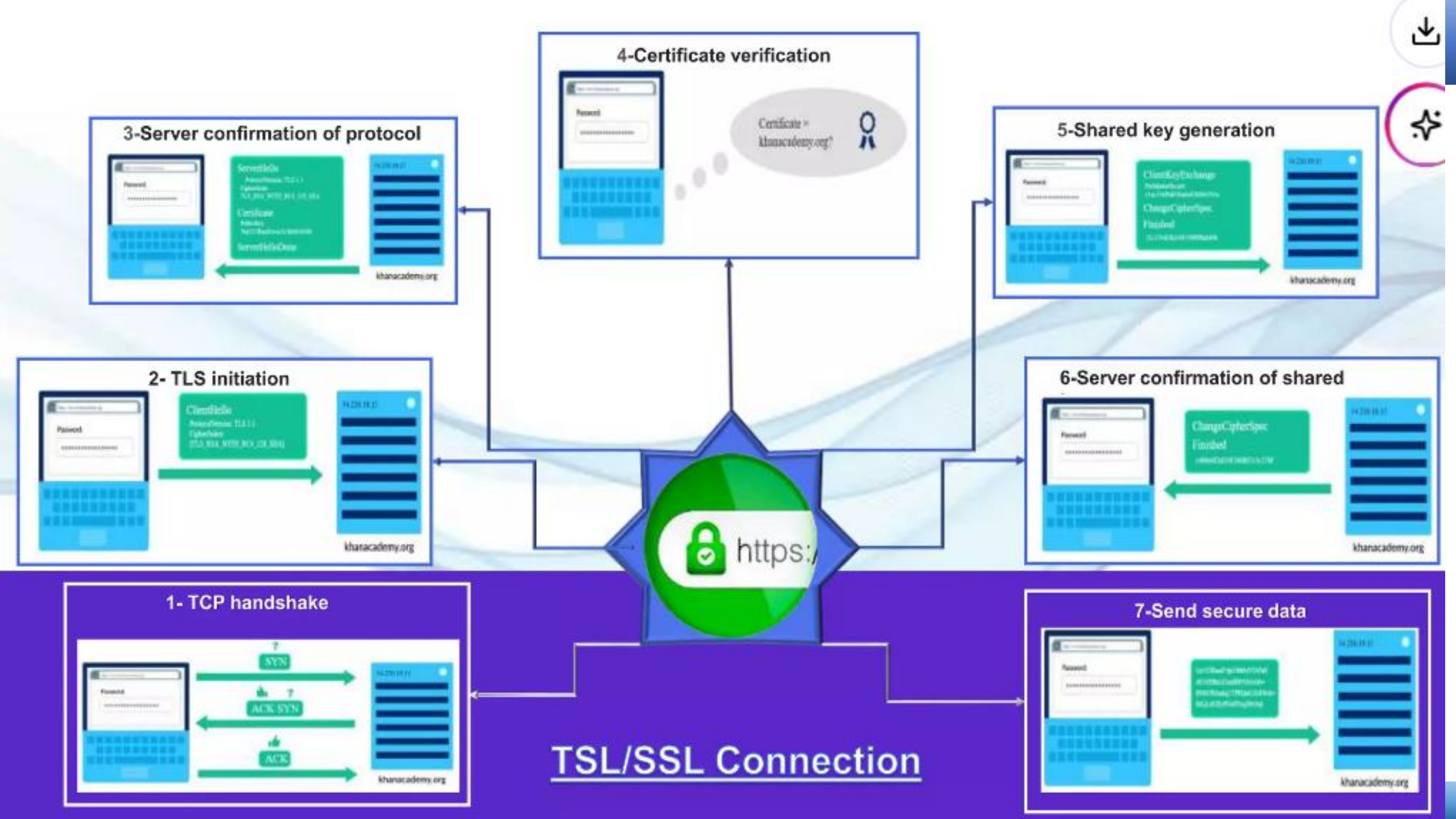


• Transport LayerSecurity (TLS)

- Secure Sockets Layer (SSL)
- TLS/SSL Protocols adds a layer of security on top of the TCP/IP transport protocols. they uses both symmetric encryption and public key encryption for securely sending private data, and adds additional security features, such as authentication and message tampering detection.
- Are standard security technology for establishing an encrypted link between a server and a client—typically a web server (website) and a browser, or a mail server and a mail client (e.g., Outlook).

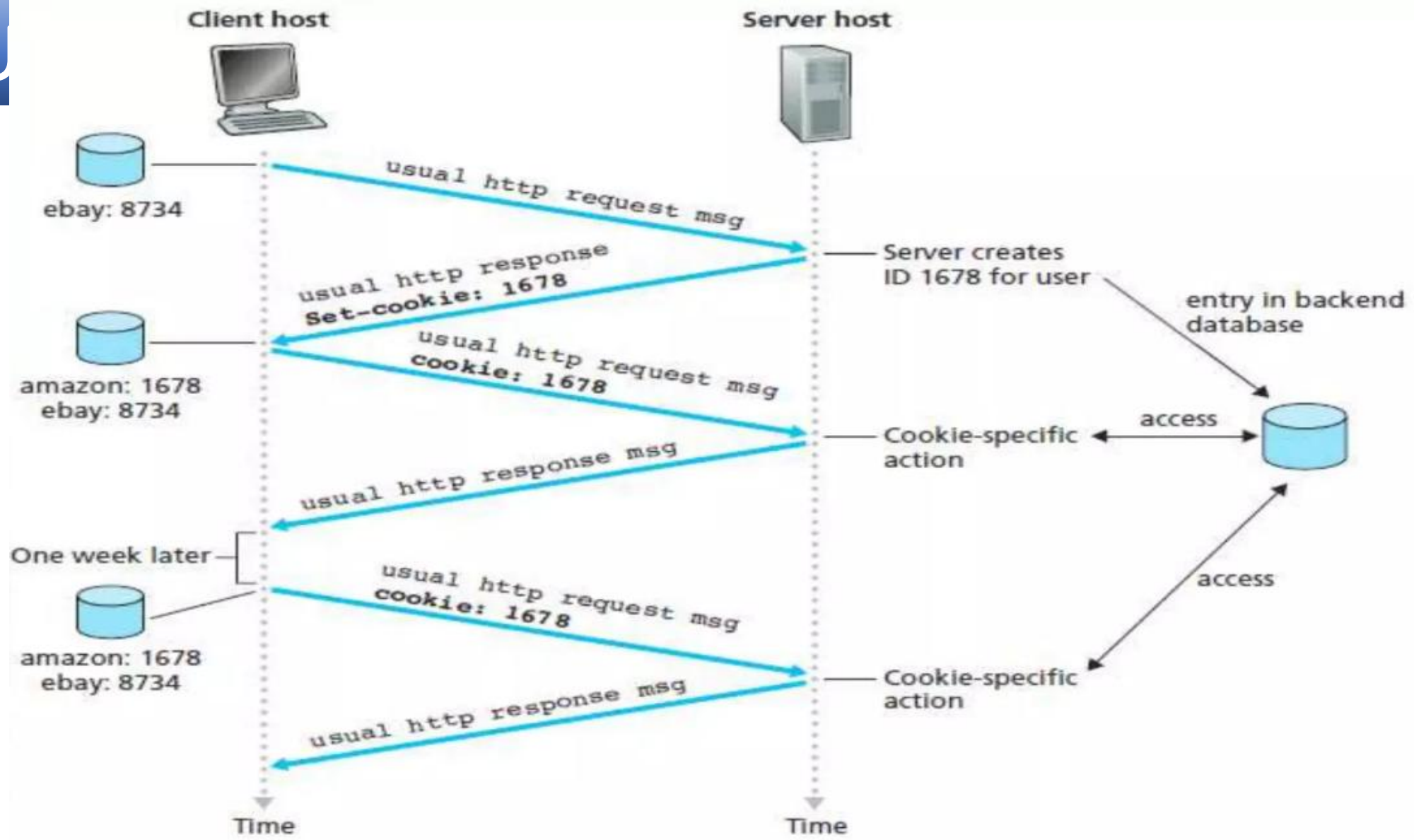
How SSL Overcomes HTTP Security Concerns

- Secure Sockets Layer technology protects your Web site and makes it easy for your Web site visitors to trust you in three essential ways:
  - Privacy
    - An SSL Certificate enables **encryption** of sensitive information during online transactions.
  - Integrity.
    - A Certificate Authority **verifies** the identity of the certificate owner when it is issued.
  - Authentication.
    - Each SSL Certificate contains unique, **authenticated** information about the certificate owner.


4-Certificate verification
Certificate =
khanacademy.org*
3-Server confirmation of protocol
ServerHello
Certificate
ServerHelloDone
khanacademy.org
5-Shared key generation
ClientKeyExchange
ChangeCipherSpec
Finished
khanacademy.org
2- TLS initiation
ClientHello
khanacademy.org
https:/
6-Server confirmation of shared
ChangeCipherSpec
Finished
khanacademy.org
1- TCP handshake
SYN
ACK SYN
ACK
khanacademy.org
7-Send secure data
khanacademy.org
TSL/SSL Connection

USER-SERVER INTERACTIONS: COOKIES

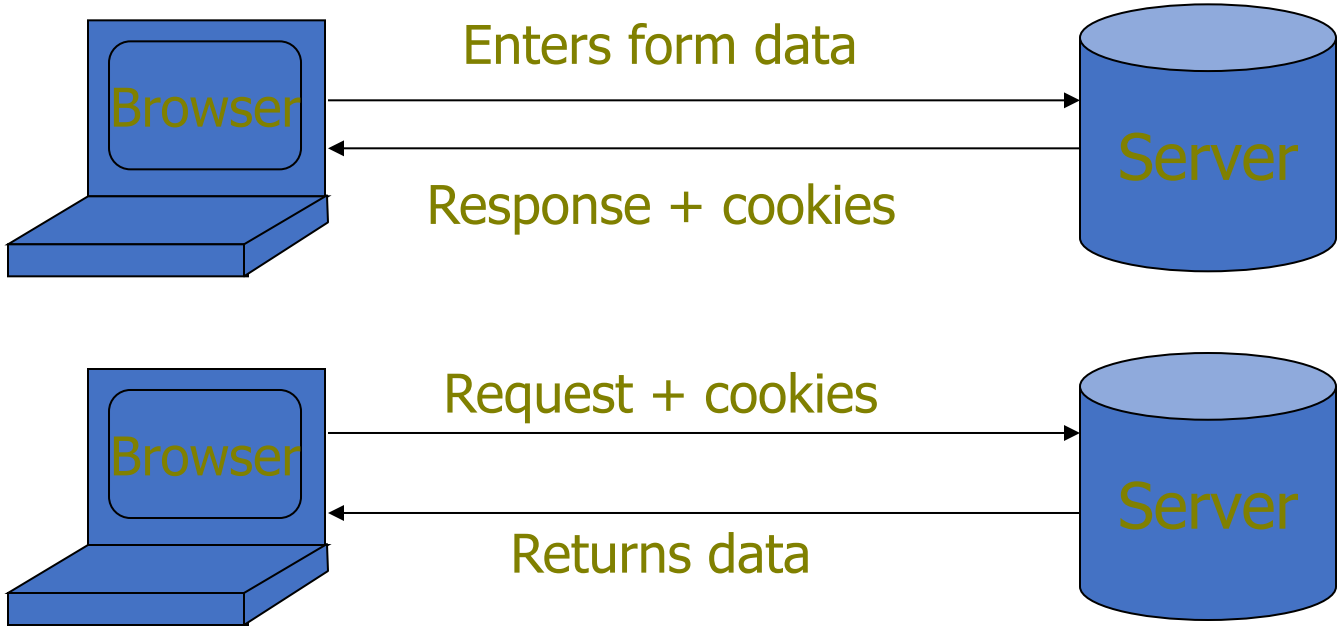
- HTTP server being stateless, simplifies server design and has permitted engineers to develop high-performance Web servers that can handle thousands of simultaneous TCP connections. However, it is often desirable for a Web site to identify users, either because the server wishes to restrict user access or because it wants to serve content as a function of the user identity.
- For these purposes, HTTP uses cookies. Cookies allow sites to keep track of users.
- The cookie technology has four components:
  - a cookie header line in the HTTP response message
  - a cookie header line in the HTTP request message
  - a cookie file kept on the user's end system and managed by the user's browser
  - a back-end database at the Web site



Keeping user state with cookies

Use Cookies to Store State Info

- Cookies
  - A cookie is a name/value pair created by a website to store information on your computer



Http is stateless protocol; cookies add state

Cookies Fields

- An example cookie from my browser
  - Name session-token
  - Content "s7yZiOvFm4YymG….”
  - Domain .amazon.com
  - Path /
  - Send For Any type of connection
  - Expires Monday, September 08, 2031 7:19:41 PM

Cookies

- Stored by the browser
- Used by the web applications
  - used for authenticating, tracking, and maintaining specific information about users
    - e.g., site preferences, contents of shopping carts
  - data may be sensitive
  - may be used to gather information about specific users
- Cookie ownership
  - Once a cookie is saved on your computer, only the website that created the cookie can read it

Web Authentication via Cookies

- HTTP is stateless
  - How does the server recognize a user who has signed in?

- Servers can use cookies to store state on client
  - After client successfully authenticates, server computes an authenticator and gives it to browser in a cookie
    - Client cannot forge authenticator on his own (session id)
  - With each request, browser presents the cookie
  - Server verifies the authenticator

How Cookies are implemented

- Cookies are sent from the server to the client via “Set-Cookie” headers

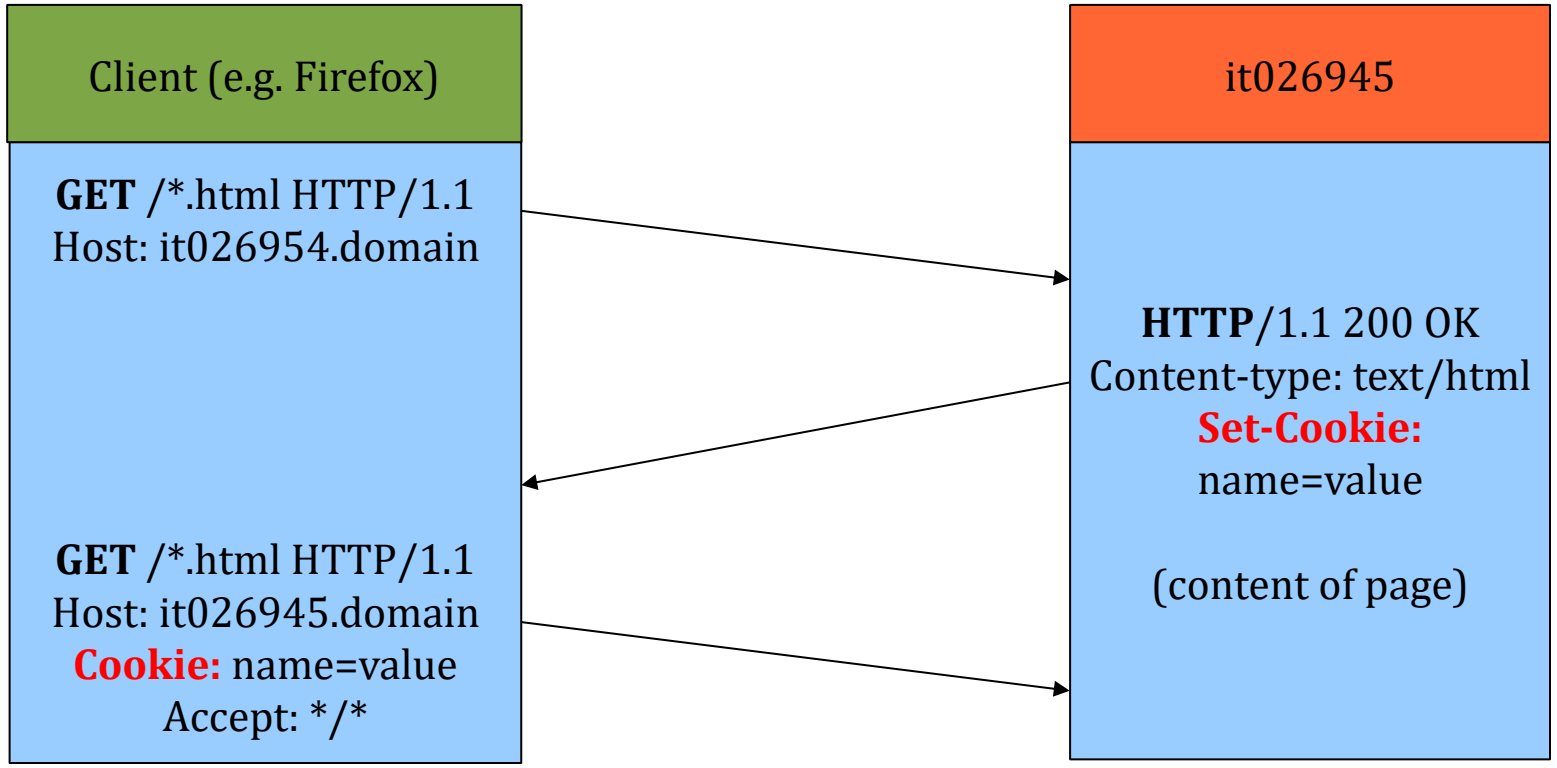
Set-Cookie: NAME=VALUE; expires=DATE; path=PATH; domain=DOMAIN_NAME; secure

- The **NAME** value is a URL-encoded name that identifies the cookie.
- The **PATH** and **DOMAIN** specify where the cookie applies

setcookie(name,value,expire,path,domain,secure)

Parameter	Description
name	(Required). Specifies the name of the cookie
value	(Required). Specifies the value of the cookie
expire	(Optional). Specifies when the cookie expires. e.g. **time()+3600*24*30** will set the cookie to expire in **30 days**. If this parameter is not set, the cookie will expire at the end of the session (when the browser closes).
path	(Optional). Specifies the server path of the cookie. If set to "**/**", the cookie will be available within the entire domain. If set to "**/phptest/**", the cookie will only be available within the test directory and all sub-directories of **phptest**. The default value is the current directory that the cookie is being set in.
domain	(Optional). Specifies the domain name of the cookie. To make the cookie available on all subdomains of example.com then you'd set it to ".example.com". Setting it to www.example.com will make the cookie only available in the www subdomain
secure	(Optional). Specifies whether or not the cookie should only be transmitted over a secure **HTTPS** connection. **TRUE** indicates that the cookie will only be set if a secure connection exists. Default is **FALSE**.

Cookies from HTTP

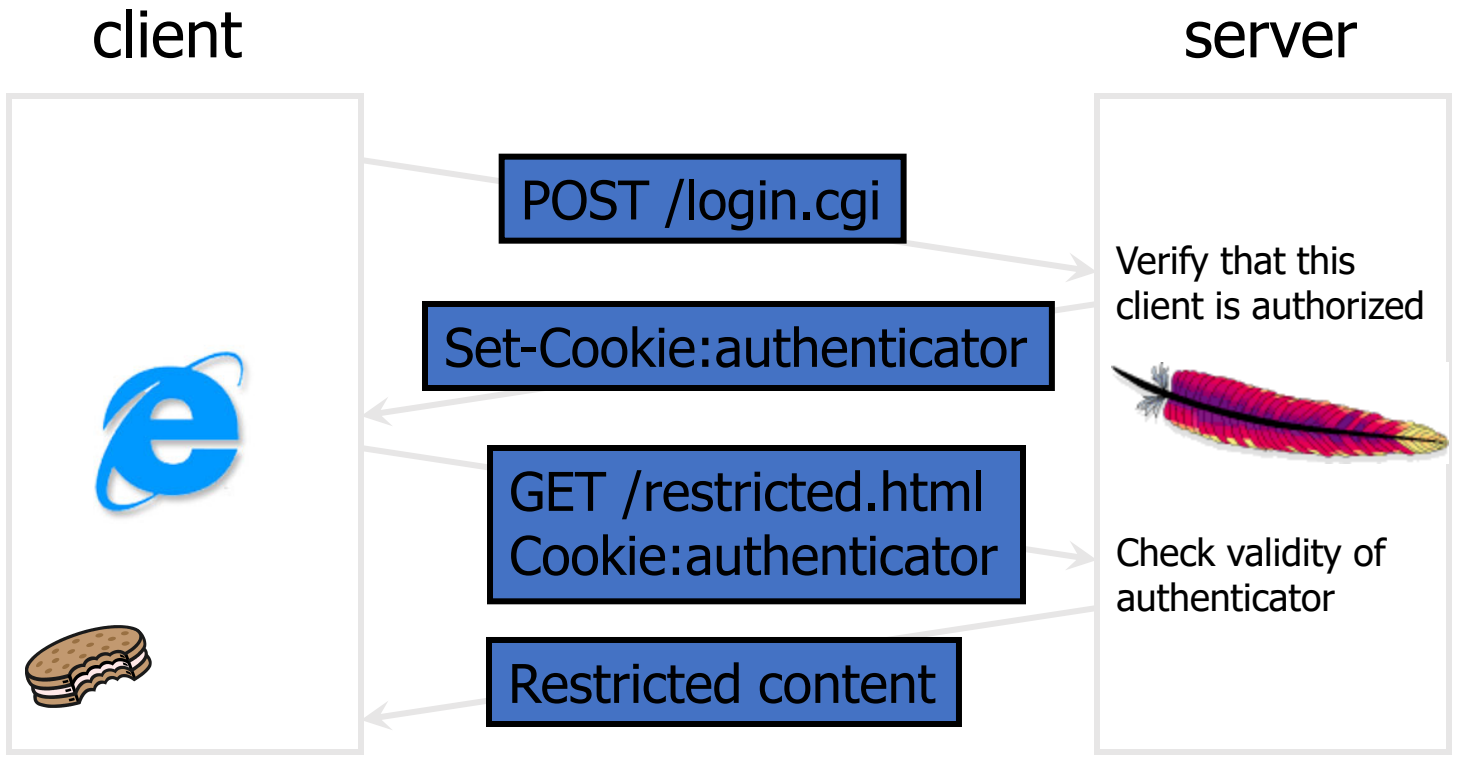


Creating PHP cookies

Cookies can be set by directly manipulating the HTTP header using the PHP header() function

```
<?php
  header("Set-Cookie: mycookie=myvalue; path=/; domain=.coggeshall.org");
?>
```

A Typical Session with Cookies



Authenticators must be unforgeable and tamper-proof

(malicious clients shouldn't be able to modify an existing authenticator)

How to design it?

Thank You